

FSI

Práctica 1.1

**Interfaz Gráfico de Análisis y Procesado
de Señales de Audio en MATLAB**

Memoria de Programación

5 de diciembre de 2024

Mario Medrano Paredes y Alonso Sandoval Martínez

Índice de contenidos

1.	Introducción	4
2.	Descripción general del código	5
2.1.	Métodos Get/Set	5
2.2.	Flujo de navegación entre pestañas	6
3.	Diseño y estructura del código.....	7
3.1.	Propiedades privadas de la clase	7
3.2.	Callbacks y eventos de la interfaz.....	8
3.3.	Gestión de errores	9
4.	Funcionalidades. Métodos principales.....	11
4.1.	Visualización.....	11
4.1.1.	Cursor temporal	11
4.1.2.	Actualización de gráficas	12
4.2.	Gestor de archivos de audio.....	13
4.2.1.	Carga y guardado de ficheros	13
4.2.2.	Limpiado de la interfaz y el reproductor	14
4.2.3.	Generador de señales	14
4.3.	Controles de audio	16
4.3.1.	Reproducción.....	16
4.3.2.	Grabación	17
4.4.	Herramientas de análisis y procesado de audio	19
4.4.1.	Suma	19
4.4.2.	Filtrado.....	20
4.4.3.	Remuestreo.....	21
4.4.4.	Modulación	22
4.4.5.	Cuantificación	23
4.4.6.	Filtro de reconstrucción	24
4.5.	Efectos y fenómenos psico-acústicos	25
4.5.1.	Ruido	25
4.5.2.	Eco	26
4.5.3.	Inversión temporal.....	27
4.5.4.	Efectos sonoros.....	28
4.5.5.	Transcripción de voz a texto	30
4.5.6.	Compresión/Expansión en el dominio temporal.....	31

4.6.	Comparación de espectrogramas	33
5.	Conclusiones.....	35
6.	Anexos.....	36
7.	Referencias	37

1. Introducción

El objetivo de esta práctica es diseñar una *Graphical User Interface (GUI)* para el procesado, grabación y reproducción de señales de audio y voz, así como su visualización en los dominios temporal y frecuencial, entre otras herramientas. Este programa está diseñado para ser completo y ofrecer numerosas utilidades, pero sin renunciar a un enfoque que pretende facilitar un diseño simple y fácil de usar para el usuario.

Se ha hecho que la potencia y versatilidad de la aplicación no interfieran con la simplicidad de su uso. Para ello, se han integrado diferentes funcionalidades que permiten realizar operaciones como la grabación, análisis espectral, manipulación de la señal en el dominio del tiempo y la frecuencia, y la visualización en tiempo real de los efectos generados.

El programa ha sido diseñado utilizando MATLAB R2024b, con el apoyo de diversos toolboxes, como App Building, Signal Processing Toolbox y Audio Toolbox, que proporcionan las herramientas necesarias para la creación de interfaces gráficas y el procesamiento avanzado de señales de audio.

En esta memoria de programación, se detallará el proceso de desarrollo de la aplicación, explicando tanto la estructura de la interfaz como los principios que guían el diseño del código. Además, se describirán los métodos principales implementados para el manejo y procesamiento de los datos, ilustrados con ejemplos y fragmentos de código comentados. A través de esta documentación, se pretende ofrecer una explicación clara y completa del proyecto, facilitando su comprensión tanto para usuarios técnicos como para aquellos interesados en aplicaciones de procesamiento de señales de audio. Finalmente, se aportarán algunas conclusiones sobre el desarrollo de la aplicación.

2. Descripción general del código

En esta sección se detalla la estructura y los componentes principales de la interfaz de usuario desarrollada, así como el flujo de navegación entre las diferentes pestañas. Además, se explican los métodos de acceso (`get` y `set`) utilizados para gestionar las propiedades privadas de la clase y la función `updateTab` que maneja el cambio de pestaña activa.

2.1. Métodos Get/Set

Los métodos `get` y `set` son fundamentales para encapsular y controlar el acceso a las propiedades privadas de la clase. Estos métodos aseguran que las propiedades internas sean manipuladas de manera controlada, manteniendo la integridad de los datos y facilitando la modularidad del código.

- **Métodos `Get`:** Permiten acceder a los valores de las propiedades privadas desde otras partes de la clase. Por ejemplo, `getAudioData` obtiene los datos de audio de la pestaña seleccionada.
- **Métodos `Set`:** Permiten asignar valores a las propiedades privadas de manera controlada. Por ejemplo, `setAudioData` asigna los datos de audio a la pestaña correspondiente.

A continuación, se detalla un ejemplo con la obtención de la frecuencia de muestreo para cada pestaña del visualizador. El método `getFs` recibe un índice de pestaña (`tabIndex`) y devuelve la frecuencia de muestreo (`Fs`) correspondiente a esa pestaña. Utiliza una estructura `switch` para determinar qué propiedad devolver en función del índice proporcionado.

```
function Fs = getFs(app, tabIndex)
    switch tabIndex
        case 0
            Fs = app.Fs0;
        case 1
            Fs = app.Fs1;
        case 2
            Fs = app.Fs2;
        case 3
            Fs = app.Fs3;
        case 4
            Fs = app.Fs4;
        case 5
            Fs = app.Fs5;
        otherwise
            Fs = [];
    end
end
```

Por otro lado, `setFs` asigna una frecuencia de muestreo (`Fs`) a la pestaña especificada por el índice (`tabIndex`). Nuevamente, se utiliza una estructura `switch` para dirigir la asignación a la propiedad correcta.

```

function Fs = getFs(app, tabIndex)
    switch tabIndex
        case 0
            Fs = app.Fs0;
        case 1
            Fs = app.Fs1;
        case 2
            Fs = app.Fs2;
        case 3
            Fs = app.Fs3;
        case 4
            Fs = app.Fs4;
        case 5
            Fs = app.Fs5;
        otherwise
            Fs = [];
    end
end

```

2.2. Flujo de navegación entre pestañas

La navegación entre las diferentes pestañas de la interfaz es gestionada por la función `updateTab`, que asegura que la aplicación responda adecuadamente a los cambios de pestaña seleccionados por el usuario. Esta función actualiza los datos y las visualizaciones correspondientes a la pestaña activa, garantizando que el usuario siempre interactúe con la información relevante.

Para ello, primero se obtiene la pestaña actual utilizando `SelectedTab` y se extrae su títulos utilizando `Title`.

```

function updateTab(app)
    app.selectedTab = app.TabGroup.SelectedTab;
    tabName = app.TabGroup.SelectedTab.Title;

```

Luego, se utiliza una estructura `switch` para cambiar entre las distintas pestañas. Para cada una de ellas, se extrae su índice utilizando `tabIndex`.

```

switch tabName
    case 'Señal Trabajo'
        app.tabIndex = 0;
    case 'Señal 1'
        app.tabIndex = 1;
    case 'Señal 2'
        app.tabIndex = 2;
    case 'Señal 3'
        app.tabIndex = 3;
    case 'Señal 4'
        app.tabIndex = 4;
    case 'Señal 5'
        app.tabIndex = 5;
    otherwise
        app.tabIndex = 0;
end

```

Una vez obtenido el índice de la pestaña actual, obtenemos los datos de audio y la frecuencia de muestreo del audio, y luego actualizamos la información de la interfaz en consecuencia.

```
% Obtener datos de audio y fs de la pestaña seleccionada
audioData = app.getAudioData(app.tabIndex);
Fs = app.getFs(app.tabIndex);

% Si hay datos de audio, actualizar la información en la interfaz
if ~isempty(audioData) && ~isempty(Fs)
    app.updateAudioInfo(audioData, Fs);
else
```

En caso de que la pestaña esté vacía, reiniciamos todos los valores.

```
% Si no hay datos, limpiar la info de la pista de audio
app.FrecMuestreoLabel.Text = '';
app.NumMuestrasLabel.Text = '';
app.NumCanalesLabel.Text = '';
app.DuracionLabel.Text = '';
app.PotenciaLabel.Text = '';
end
end
```

3. Diseño y estructura del código

En esta sección se detalla la arquitectura interna de la aplicación. Se explican las propiedades privadas, los callbacks y eventos de la interfaz, así como la gestión de errores implementada. A continuación, se profundiza en cada una de estas áreas.

3.1. Propiedades privadas de la clase

Las propiedades de la clase en MatLab son variables que almacenan el estado y los datos necesarios para el funcionamiento de la aplicación. En este proyecto, se han definido varias propiedades privadas que gestionan diferentes aspectos del procesamiento y análisis de audio. A continuación, se describe brevemente cada una de estas propiedades:

```
properties (Access = private)
    y; % Datos de audio temporal
    isPlaying; % Indicador de reproduccion

    % Cursores para cada pestaña
    cursorLine0; cursorLine1; cursorLine2;
    cursorLine3; cursorLine4; cursorLine5;

    % Datos de audio para cada pestaña
    audioData0; audioData1; audioData2;
    audioData3; audioData4; audioData5;

    % Frecuencias de muestreo
    Fs0; Fs1; Fs2; Fs3; Fs4; Fs5;
```

```

% Reproductores de audio
player0; player1; player2; player3; player4; player5;

accumulatedSignal;      % Señal acumulada
FsAccumulated;          % Frecuencia de muestreo acumulada

selectedTab;            % Pestaña seleccionada
tabIndex;               % Índice

recorder;               % Audiorecorder
isRecording = false;    % Indicador de grabacion
recordedAudioData;      % Datos de audio grabados
sampleRate = 44100;     % Frecuencia de muestreo grabacion

bearer = '***';
end

```

- La variable `y` guarda temporalmente los datos de audio a reproducir
- `isPlaying` indica si hay una reproducción en curso
- `cursorLine` guarda la posición del cursor que indica el instante del audio por el que nos llegamos en la gráfica respecto al tiempo. Hay una de estas variables para cada pestaña
- `audioData`, `Fs`, y `player` guardan respectivamente los datos de audio, la frecuencia de muestreo y los reproductores para cada pestaña
- `accumulatedSignal` y `FsAccumulated` se emplean en la suma de señales
- `selectedTab` y `tabIndex` sirven para la gestión de las pestañas en el visualizador
- `recorder`, `isRecording`, `recordedAudioData` y `sampleRate` se usan en el apartado de grabación de audio en tiempo real
- `bearer` es el token de autenticación para la API de Hugging Face, que permite implementar la transcripción de audio con IA como veremos más tarde

3.2. Callbacks y eventos de la interfaz

Los callbacks y eventos son fundamentales en la interacción entre el usuario y la aplicación desarrollada en MatLab. Estos mecanismos permiten que la aplicación responda de manera dinámica a las acciones realizadas por el usuario, como pulsaciones de botones, cambios en menús desplegables, o modificaciones en campos de entrada. Cada componente interactivo de la interfaz está asociado a uno o más callbacks que definen el comportamiento de la aplicación ante dichos eventos.

Función de callback

Es una función que se ejecuta automáticamente en respuesta a un evento específico en la interfaz de usuario. Por ejemplo, cuando se pulsa el botón de reproducción, se activa el callback asociado que gestiona la reproducción del audio seleccionado.

```

% Button pushed function: PlayButton
function PlayButtonPushed(app, event)
    playAudio(app);
end

```


Manejo de Cambios en la Interfaz

Además de las acciones de botones, los callbacks también gestionan cambios en otros componentes, como menús desplegables o campos de texto. Estos permiten actualizar dinámicamente la configuración de la aplicación según las preferencias del usuario.

En el siguiente ejemplo, el callback se activa cuando el usuario cambia la selección en el menú desplegable de efectos. Dependiendo del tipo seleccionado, se habilitan o deshabilitan ciertos campos de entrada, adaptando la interfaz a las necesidades específicas de cada tipo de efecto.

```
% Value changed function: ElegirefectoListBox
function ElegirefectoListBoxValueChanged(app, event)
    efectoSeleccionado = app.ElegirefectoListBox.Value;

    % Habilitar/deshabilitar campos según el efecto seleccionado
    switch efectoSeleccionado
        case 'Reverberación'
            app.DuracinsEditField.Enable = 'on';
            app.DuracinsEditFieldLabel.Enable = 'on';
            app.DesfasedecanalHaasEditField.Enable = 'off';
            app.DesfasedecanalHaasEditFieldLabel.Enable = 'off';

        case 'Efecto Haas'
            app.DesfasedecanalHaasEditField.Enable = 'on';
            app.DesfasedecanalHaasEditFieldLabel.Enable = 'on';
            app.DuracinsEditField.Enable = 'off';
            app.DuracinsEditFieldLabel.Enable = 'off';

        otherwise
            app.DuracinsEditField.Enable = 'off';
            app.DuracinsEditFieldLabel.Enable = 'off';
            app.DesfasedecanalHaasEditField.Enable = 'off';
            app.DesfasedecanalHaasEditFieldLabel.Enable = 'off';

    end
end
```

3.3. Gestión de errores

La gestión de errores es esencial para asegurar la robustez y fiabilidad de la aplicación desarrollada en MATLAB. En este proyecto, se implementan mecanismos que permiten detectar, capturar y manejar situaciones inesperadas, garantizando que la aplicación responda de manera adecuada sin interrumpir la experiencia del usuario. A continuación, se presentan dos mecanismos de control de errores utilizados en nuestra aplicación.

Bloques try-catch

```
% Cargar un archivo de audio en la pestaña activa
function loadAudio(app)
    [file, path] = uigetfile('*.wav', 'Select an audio file');
    if isequal(file, 0)
        return;
    end
```

```

        fullPath = fullfile(path, file);
        app.EditField.Value = fullPath; % Mostrar la ruta del archivo
        app.clearAudio();
        try
            [y, Fs] = audioread(fullPath);
            app.updateTab();

            % ... (resto del código)

        catch ME
            errordlg(['Error loading audio file: ' ME.message], 'Error');
            fprintf('Error details: %s\n', getReport(ME));
        end
    end
end

```

En este ejemplo, si ocurre un error durante la lectura del archivo de audio, el bloque `catch` captura la excepción y muestra un cuadro de diálogo informando al usuario, además de registrar los detalles del error en la consola para facilitar la depuración.

Alertas de usuario (`uialert` y `errordlg`)

```

% Guardar audio
function saveAudio(app)
    audioData = app.getAudioData(app.tabIndex);
    Fs = app.getFs(app.tabIndex);
    if isempty(audioData)
        uialert(app.UIFigure, 'No hay datos de audio para guardar.',
'Error');
    else
        [file, path] = uiputfile('*.wav', 'Save audio file');
        if ~isequal(file, 0)
            fullFileName = fullfile(path, file);
            audiowrite(fullFileName, audioData, Fs);
            app.EditField.Value = fullFileName;
            uialert(app.UIFigure, ['Archivo guardado: ' fullFileName],
'Success');
        end
    end
end
end

```

Se emplean funciones como `uialert` y `errordlg` para notificar al usuario sobre errores o estados importantes, mejorando la interacción y proporcionando retroalimentación inmediata. Aquí, si el usuario no proporciona datos de audio para guardar, se muestra una alerta indicando el problema, evitando que la aplicación proceda con parámetros incorrectos.

4. Funcionalidades. Métodos principales

4.1. Visualización

La visualización es una parte crucial de la aplicación, ya que permite al usuario interpretar y analizar las señales de audio de manera intuitiva. La aplicación ofrece tres tipos principales de visualizaciones: Dominio Temporal, Dominio Frecuencial y Espectrograma. Cada una de estas visualizaciones proporciona una perspectiva diferente de la señal de audio, facilitando el análisis detallado y la identificación de características específicas.

4.1.1. Cursor temporal

Se emplea la siguiente función para actualizar en todo momento la posición del cursor e ir dibujándolo sobre la gráfica en el dominio temporal. Simplemente se obtiene la posición actual en segundos gracias a la frecuencia de muestreo y se va desplazando el cursor a lo largo del tiempo usando set.

```
% Función para actualizar el cursor en la gráfica
function updateCursor(app)
    app.updateTab();
    tabIndex = app.tabIndex;
    player = app.getPlayer(tabIndex);
    Fs = app.getFs(tabIndex); % Frecuencia de muestreo correcta
    cursorLine = app.getCursorLine(tabIndex);
    timeAxes = app.getTimeAxes(tabIndex);

    if ~isempty(player) && isplaying(player)
        % Calcular el tiempo actual en segundos
        currentTime = player.CurrentSample / Fs;

        % Solo mover el cursor sin redibujar la señal
        if isempty(cursorLine) || ~isvalid(cursorLine)
            % Si la línea del cursor no existe, la creamos
            cursorLine = plot(timeAxes, [currentTime currentTime],
ylim(timeAxes), 'r', 'LineWidth', 1.5);
            app.setCursorLine(tabIndex, cursorLine);
        else
            % Si la línea del cursor ya existe, solo la actualizamos
            set(cursorLine, 'XData', [currentTime currentTime]);
        end
    else
        % Si no se está reproduciendo, ocultamos el cursor
        if ~isempty(cursorLine) && isvalid(cursorLine)
            set(cursorLine, 'XData', [NaN NaN]);
        end
    end
end
```

4.1.2. Actualización de gráficas

La función `updatePlot` es responsable de actualizar las tres gráficas principales de la aplicación: dominio temporal, dominio frecuencial y espectrograma. La lógica seguida asegura que cada gráfica refleje correctamente el estado actual de la señal de audio seleccionada.

- Obtención de Datos: Recupera los datos de audio y la frecuencia de muestreo de la pestaña activa.
- Conversión a Mono: Si el audio es multicanal, se convierte a mono promediando los canales.

Dominio Temporal: Utiliza `plot` para visualizar la amplitud de la señal en función del tiempo.

Dominio Frecuencial: Calcula la FFT mediante `fft` para obtener el espectro y utiliza `plot` para mostrar la señal en el eje de frecuencias.

Espectrograma: Emplea `spectrogram` e `imagesc` para representar la variación de frecuencia a lo largo del tiempo.

```
function updatePlot(app, timeAxes, freqAxes, specAxes)
    % . . . (Resto del código)

    % Graficar señal en el tiempo
    t = (0:length(audioData)-1) / Fs; % Eje temporal correcto
    plot(timeAxes, t, audioData);
    title(timeAxes, 'Señal - Eje temporal');
    xlabel(timeAxes, 'Tiempo (s)');
    ylabel(timeAxes, 'Amplitud');
    hold(timeAxes, 'on');

    % Graficar espectro de frecuencia
    L = length(audioData);
    Y = fft(audioData);
    P2 = abs(Y / L);
    P1 = P2(1:floor(L / 2) + 1);
    P1(2:end - 1) = 2 * P1(2:end - 1);
    f = Fs * (0:(L / 2)) / L;
    plot(freqAxes, f, P1);
    title(freqAxes, 'Señal - Espectro de Frecuencia');
    xlabel(freqAxes, 'Frecuencia (Hz)');
    ylabel(freqAxes, 'Amplitud');

    % Graficar espectrograma
    [S, F, T, P] = spectrogram(audioData, 256, 250, 256, Fs,
'yaxis');

    imagesc(specAxes, T, F, 10*log10(P));
    axis(specAxes, 'xy');
    title(specAxes, 'Espectrograma');
    xlabel(specAxes, 'Tiempo (s)');
    ylabel(specAxes, 'Frecuencia (Hz)');
end
```

4.2. Gestor de archivos de audio

El Gestor de Archivos de Audio es una de las funcionalidades fundamentales de la aplicación, encargada de manejar la carga, el guardado y la generación de señales de audio. Este gestor permite al usuario interactuar con archivos de audio existentes, así como crear nuevas señales desde cero, facilitando así el análisis y procesamiento posterior.

4.2.1. Carga y guardado de ficheros

En primer lugar, la función `updateAudioInfo` se encarga de obtener la información sobre un fichero de audio cargado y luego actualiza las etiquetas correspondientes:

```
% Obtener detalles de la pista de audio
numMuestras = size(audioData, 1);
numCanales = size(audioData, 2);
duracion = numMuestras / Fs;
potencia = rms(audioData)^2;
```

Si el audio tiene más de un canal, se convierte en audio mono previamente a calcular su información:

```
if size(audioData, 2) > 1
    audioData = mean(audioData, 2);
end
```

La función `loadAudio` es la encargada de permitir al usuario cargar ficheros en formato `.wav` y reproducirlos en la aplicación. Para ello, se utiliza `uigetfile` para seleccionar un fichero en nuestro explorador de archivos y luego se lee el audio con `audioread`. Finalmente, se actualiza el reproductor, la información del audio y las tres gráficas.

```
% Cargar un archivo de audio en la pestaña activa
function loadAudio(app)
    [file, path] = uigetfile('*.wav', 'Select an audio file');
    if isequal(file, 0)
        return;
    end
    fullPath = fullfile(path, file);
    app.EditField.Value = fullPath; % Mostrar la ruta del archivo
    app.clearAudio();
    try
        [y, Fs] = audioread(fullPath);
        app.updateTab();

        app.setAudioData(app.tabIndex, y);
        app.setFs(app.tabIndex, Fs);

        updateAudioInfo(app, y, Fs);

        timeAxes = app.getTimeAxes(app.tabIndex);
        freqAxes = app.getFreqAxes(app.tabIndex);
        specAxes = app.getSpecAxes(app.tabIndex);
        app.updatePlot(timeAxes, freqAxes, specAxes);

    catch ME
```

```

        errordlg(['Error loading audio file: ' ME.message], 'Error');
        fprintf('Error details: %s\n', getReport(ME));
    end
end

```

Para guardar el audio como un fichero .wav en la máquina del usuario, se emplea la función `saveAudio`. Esta función usa `audiowrite` para escribir el audio en un nuevo archivo.

```

fullFileName = fullfile(path, file);
audiowrite(fullFileName, audioData, Fs);
app.EditField.Value = fullFileName;
uialert(app.UIFigure, ['Archivo guardado: ' fullFileName], 'Success');

```

4.2.2. Limpiado de la interfaz y el reproductor

Para asegurar que nuevas cargas de audio no interfieran con datos anteriores, la función de limpieza `clearAudio` restablece la interfaz gráfica y detiene cualquier reproducción activa. Esto garantiza que la aplicación esté preparada para manejar nuevas señales de manera eficiente.

```

% Detener la reproducción y limpiar el reproductor
player = app.getPlayer(app.tabIndex);
if ~isempty(player) && isvalid(player)
    stop(player);
    app.setPlayer(app.tabIndex, []); % Limpiar el objeto player
end

% Limpiar las gráficas de la pestaña activa
timeAxes = app.getTimeAxes(app.tabIndex);
freqAxes = app.getFreqAxes(app.tabIndex);
specAxes = app.getSpecAxes(app.tabIndex);
cla(timeAxes);
cla(freqAxes);
cla(specAxes);

```

Además, esta función también se encarga de resetear todas las etiquetas que contienen información sobre el audio, así como de limpiar el reproductor.

4.2.3. Generador de señales

La funcionalidad de generar señales se implementa a través del método `generateSignal` permite al usuario crear ondas de diferentes tipos (senoidal, cuadrada, triangular, dientes de sierra) con parámetros personalizables como amplitud, frecuencia y duración.

Lo primero es actualizar la pestaña activa y limpiar los datos previos antes de generar la nueva señal. Luego, se obtienen los parámetros desde los campos de texto de la interfaz gráfica:

```
function generateSignal(app)
    % Actualizar la pestaña activa
    updateTab(app);
    tabIndex = app.tabIndex; % Obtener el índice
    clearAudio(app);

    tipo = app.TipoDropDown.Value;
    amp = app.AmplitudVEditField.Value;
    freq = app.FEditField.Value;
    Fs = app.FsEditField.Value;
    duracion = app.DuracionsEditField.Value;
    dutyCycle = app.DutycycleEditField.Value;
    t = 0:1/Fs:duracion;
```

A continuación, se genera la señal en función del tipo seleccionado. Para ello, se emplea una estructura `switch` y posteriormente se usan las funciones `sin`, `square` y `sawtooth` para crear las distintas formas de onda.

```
% Generar la señal según el tipo seleccionado
switch tipo
    case 'Seno'
        y = amp * sin(2 * pi * freq * t);
    case 'Cuadrada'
        y = amp * square(2 * pi * freq * t, dutyCycle * 100);
    case 'Triangular'
        y = amp * sawtooth(2 * pi * freq * t, 0.5);
    case 'Dientes de sierra'
        y = amp * sawtooth(2 * pi * freq * t);
    otherwise
        y = zeros(size(t)); % Caso predeterminado, señal vacía
end
```

Posteriormente, La señal generada se guarda temporalmente en un archivo `.wav` utilizando `audiowrite`. Esto permite que la señal sea manejada de manera consistente con otras funcionalidades de la aplicación que operan sobre archivos de audio.

```
% Guardar la señal generada en un archivo temporal
tempFile = fullfile(tempdir, 'generatedSignal.wav');
audiowrite(tempFile, y, Fs);
```

Finalmente, se actualizan las visualizaciones de la señal en los ejes temporales, de frecuencia y espectrograma mediante `updatePlot`. El bloque `try-catch` garantiza que cualquier error durante la lectura o actualización sea manejado adecuadamente, informando al usuario mediante un diálogo de error.

4.3. Controles de audio

El módulo de Controles de Audio gestiona la reproducción, pausa y detención de las señales de audio dentro de la aplicación, así como la grabación en tiempo real de la voz del usuario. Este componente es fundamental para permitir a los usuarios escuchar las señales procesadas y verificar los efectos aplicados en tiempo real.

4.3.1. Reproducción

La funcionalidad de Reproducción se implementa a través del método `playAudio`. Este método gestiona la reproducción de la señal de audio seleccionada en la pestaña activa, asegurando una experiencia de usuario fluida y eficiente.

Se verifica si ya existe un objeto `audioplayer` válido para la pestaña activa. Si no es así, se crea uno nuevo. Esta práctica optimiza el uso de recursos al reutilizar objetos existentes cuando es posible.

```
% Función para reproducir el audio
function playAudio(app)
    app.updateTab();
    tabIndex = app.tabIndex;

    % Obtener datos de audio y fs para la pestaña activa
    audioData = app.getAudioData(tabIndex);
    Fs = app.getFs(tabIndex);

    if ~isempty(audioData)
        % Crear o reiniciar el reproductor para la pestaña activa
        player = app.getPlayer(tabIndex);
        if isempty(player) || ~isvalid(player)
            player = audioplayer(audioData, Fs);
            app.setPlayer(tabIndex, player);
        end
    end
```

Se utiliza la función `isplaying` para determinar el estado actual del reproductor. Si el audio está pausado, se reanuda desde la última posición; de lo contrario, se inicia la reproducción desde el punto actual.

```
% Reanudar si el audio está pausado
if isplaying(player)
    resume(player);
else
    play(player, player.CurrentSample);
end
```

Se implementa un temporizador (`timerObj`) para actualizar el cursor de tiempo en las gráficas mientras se reproduce el audio. Este enfoque asegura que la interfaz refleje en tiempo real la posición actual de la reproducción, mejorando la visualización y la interacción del usuario.


```

        if isempty(player.UserData) || ~isvalid(player.UserData)
            timerObj = timer('TimerFcn', @(~,~) app.updateCursor(),
...
                                'Period', 0.1, 'ExecutionMode', 'fixedRate');
            start(timerObj);
            player.UserData = timerObj;
        end
    end
end

```

Para pausar la reproducción, se emplea el método `pauseAudio`, que utiliza la función `pause` de Matlab.

```

player = app.getPlayer(tabIndex);
if ~isempty(player) && isplaying(player)
    pause(player);
end

```

Por último, para pausar la reproducción se utiliza el método `stopAudio` junto con la función `stop` de Matlab. Además, se elimina el temporizador del cursor.

```

player = app.getPlayer(tabIndex);
if ~isempty(player) && isvalid(player)
    stop(player);

    % Detener y eliminar el temporizador del cursor
    if isvalid(player.UserData)
        stop(player.UserData);
        delete(player.UserData);
        player.UserData = [];
    end
end

```

4.3.2. Grabación

La funcionalidad de Grabación permite a los usuarios capturar audio directamente desde un micrófono, almacenar los datos grabados y visualizarlos en la interfaz. Este módulo se implementa mediante los métodos `startRecording` y `stopRecording`.

Se configura el objeto `audiorecorder` con una profundidad de bits de 16 y una grabación monoaural:

```

function startRecording(app)
    nBits = 16;
    nChannels = 1;
    app.recorder = audiorecorder(app.sampleRate, nBits, nChannels);

```

Al iniciar la grabación, se activa una lámpara indicadora (`GrabandoLamp`) en color rojo para alertar al usuario de que la grabación está en curso. Además, se establece el indicador `isRecording` para controlar el ciclo de grabación.

```

% Ponemos el LED en rojo al empezar a grabar
record(app.recorder);
app.GrabandoLamp.Color = [1 0 0];
app.GrabandoLamp.Enable = 1;
app.isRecording = true;

```

Durante la grabación, se actualiza continuamente la gráfica de la señal de audio en el dominio temporal.

```

% Graficamos el audio en tiempo real
while app.isRecording
    pause(0.1);
    y = getaudiodata(app.recorder);
    plot(app.UIAxes_Time, y);
    drawnow;
end
end

```

Al detener la grabación, se recuperan los datos grabados y se desactiva la lámpara indicadora.

```

% Funcion para detener la grabacion
function stopRecording(app)
    if app.isRecording
        stop(app.recorder);
        app.recordedAudioData = getaudiodata(app.recorder);
        app.isRecording = false;

        % Apagamos el LED
        app.GrabandoLamp.Color = [0.5 0.5 0.5];
        app.GrabandoLamp.Enable = 0;
    end
end

```

Los datos grabados se asignan a la pestaña activa, y se actualizan todas las visualizaciones correspondientes. Además, se configura un nuevo objeto `audioplayer` para permitir la reproducción

```

% Cargamos el audio grabado en la pestaña activa
app.updateTab();
app.setAudioData(app.tabIndex, app.recordedAudioData);
app.setFs(app.tabIndex, app.sampleRate);

% Actualizamos los plots
timeAxes = app.getTimeAxes(app.tabIndex);
freqAxes = app.getFreqAxes(app.tabIndex);
specAxes = app.getSpecAxes(app.tabIndex);
app.updatePlot(timeAxes, freqAxes, specAxes);

% Creamos el reproductor de audio
player = audioplayer(app.recordedAudioData, app.sampleRate);
app.setPlayer(app.tabIndex, player);
end
end

```

4.4. Herramientas de análisis y procesamiento de audio

Este módulo integra una serie de funcionalidades avanzadas que permiten a los usuarios modificar y analizar las señales de audio de diversas maneras. Estas herramientas son fundamentales para aplicar transformaciones como suma, filtrado, remuestreo, modulación, cuantificación y otros procesos que enriquecen el análisis y manipulación de las señales. A continuación, se detallan las principales funcionalidades implementadas, explicando su diseño y justificación.

4.4.1. Suma

La interfaz nos permite la suma de señales de audio, así como la acumulación progresiva de varias señales en una misma ranura del visualizador. Esta funcionalidad está implementada mediante los métodos `accumulateSignals` y `clearAccumulatedSignals`.

Para sumar las señales, simplemente se actualiza la señal acumulada como la señal anterior más la señal actual:

```
% Asegurarse de que las señales tengan la misma fs
if app.FsAccumulated ~= Fs
    uialert(app.UIFigure, 'Las frecuencias de muestreo de las
señales no coinciden.', 'Error');
    return;
end
% Asegurar que ambas señales tengan el mismo tamaño
minLength = min(length(app.accumulatedSignal),
length(audioData));
app.accumulatedSignal = app.accumulatedSignal(1:minLength) +
audioData(1:minLength);
```

Luego, se actualizan los ejes para mostrar la nueva señal acumulada como ya hemos hecho en otros métodos.

```
% Mostrar la señal acumulada en la pestaña de "Señal de trabajo"
timeAxes = app.UIAxes_Time;
freqAxes = app.UIAxes_Frequency;
specAxes = app.UIAxes_Espectrogram;
app.updatePlot(timeAxes, freqAxes, specAxes);
```

Para eliminar el total de las señales acumuladas, simplemente limpiamos las variables y la pestaña de trabajo en el visualizador.

```
function clearAccumulatedSignals(app)
% Limpiar la señal acumulada y su frecuencia de muestreo
app.accumulatedSignal = [];
app.FsAccumulated = [];

% Limpiar la pestaña de "Señal de trabajo"
cla(app.UIAxes_Time);
cla(app.UIAxes_Frequency);
cla(app.UIAxes_Espectrogram);
end
```

4.4.2. Filtrado

Esta función permite a los usuarios aplicar filtros digitales a las señales de audio para eliminar o atenuar ciertas frecuencias. Esto se implementa a través del método `applyFilter`, que diseña y aplica filtros de diferentes tipos según las especificaciones del usuario.

Los parámetros necesarios para el diseño del filtro se obtienen directamente desde los componentes de la interfaz de usuario.

```
% Recuperar parámetros del filtro desde la GUI
tipoFiltro = app.TipodefiltroDropDown.Value;
orden = app.OrdenNEditField.Value;
freqCorte = str2double(app.FrecCorteHzEditField.Value);
freqCorteInferior = str2double(app.FrecCorteInferiorHzEditField.Value);
freqCorteSuperior = str2double(app.FrecCorteSuperiorHzEditField.Value);
```

Para diseñar el filtro, se utiliza la función `butter` de Matlab para diseñar filtros Butterworth de orden especificado y frecuencia de corte normalizada. A continuación podemos ver un ejemplo para el primer tipo de filtro, el paso bajo.

```
% Diseñar el filtro según el tipo seleccionado
switch tipoFiltro
case 'Paso Bajo'
    if isempty(freqCorte)
        uialert(app.UIFigure, 'Es necesario especificar la frecuencia de corte para un filtro paso bajo.', 'Error');
        return;
    end
    [b, a] = butter(orden, freqCorte / (Fs / 2), 'low');
```

La función `filtfilt` aplica el filtro en ambas direcciones, eliminando el retardo de fase y asegurando una señal procesada de alta calidad. La normalización evita la distorsión causada por el aumento de amplitud tras el filtrado, y la señal filtrada se actualiza en las propiedades internas de la aplicación.

```
% Aplicar el filtro a la señal
audioProcesado = filtfilt(b, a, audioData);

% Normalizar para evitar distorsión/clipping
audioProcesado = audioProcesado / max(abs(audioProcesado));

% Actualizar la señal procesada en la pestaña actual
app.setAudioData(tabIndex, audioProcesado);
```

Finalmente, se detiene cualquier reproducción en curso, se crea un nuevo objeto `audioplayer` con la señal filtrada y se actualizan las gráficas para reflejar los cambios.

```

% Detener el reproductor existente
player = app.getPlayer(tabIndex);
if ~isempty(player) && isvalid(player)
    stop(player);
end

% Crear un nuevo reproductor con la señal filtrada
player = audioplayer(audioProcesado, Fs);
app.setPlayer(tabIndex, player);

% Actualizar las gráficas
timeAxes = app.getTimeAxes(tabIndex);
freqAxes = app.getFreqAxes(tabIndex);
specAxes = app.getSpecAxes(tabIndex);
app.updatePlot(timeAxes, freqAxes, specAxes);

```

4.4.3. Remuestreo

El remuestreo permite cambiar la frecuencia de muestreo de una señal de audio, lo que es útil para adaptarse a diferentes requisitos de procesamiento o transmisión. Este proceso se implementa mediante el método `applyResampling`.

Se obtienen los parámetros necesarios desde la interfaz de usuario, permitiendo diferentes métodos de remuestreo según las preferencias del usuario. Luego, se inicializan las variables necesarias.

```

% Recuperar los parámetros de remuestreo de la GUI
FsDestino = app.FrecMuestreodedestinoEditField.Value;
factorInterpolacion = app.FactordeinterpolacinEditField.Value;
factorDiezmado = app.FactordediezmadoEditField.Value;

% Inicializar las variables
audioProcesado = audioData;
FsNuevo = Fs;

```

Para la aplicación del remuestreo, se utiliza la función `resample` de Matlab para cambiar la frecuencia de muestreo de manera directa cuando se especifica `FsDestino`. Si por el contrario el usuario desea aplicar el remuestreo mediante un ratio, se aplican factores de interpolación y diezmado para modificar la frecuencia de muestreo, usando la función `upsample` de Matlab y agregando un filtro anti-aliasing antes del diezmado con `butter`, `filtfilt` y `downsample` de Matlab.

```

% Lógica de remuestreo
if ~isempty(FsDestino) && FsDestino > 0
    % Caso 1: Frecuencia de muestreo destino tiene prioridad
    audioProcesado = resample(audioData, FsDestino, Fs);
    FsNuevo = FsDestino;
elseif (~isempty(factorInterpolacion) && factorInterpolacion > 0)
|| ...
    (~isempty(factorDiezmado) && factorDiezmado > 0)
    % Caso 2: Usar factores de interpolación y/o diezmado
    if ~isempty(factorInterpolacion) && factorInterpolacion > 1
        audioProcesado = upsample(audioProcesado,
round(factorInterpolacion));
        FsNuevo = FsNuevo * factorInterpolacion;

```

```

        end
        if ~isempty(factorDiezmado) && factorDiezmado > 1
            % Filtro anti-aliasing antes del diezmado
            [b, a] = butter(4, 1 / factorDiezmado, 'low');
            audioProcesado = filtfilt(b, a, audioProcesado);
            audioProcesado = downsample(audioProcesado,
round(factorDiezmado));
            FsNuevo = FsNuevo / factorDiezmado;
        end
    else
        % Caso 3: Sin parámetros válidos
        uialert(app.UIFigure, 'Por favor, especifica un parámetro
válido para el remuestreo.', 'Error');
        return;
    end
end

```

Tras el remuestreo, se actualizan los datos internos y las visualizaciones. Se crea un nuevo reproductor con la señal remuestreada y se inicia la reproducción para verificar los cambios realizados. Luego se actualizan las gráficas como en el resto de los apartados.

```

% Actualizar los datos en la pestaña activa
app.setAudioData(tabIndex, audioProcesado);
app.setFs(tabIndex, FsNuevo);

% Crear un nuevo reproductor de audio con la señal procesada
player = app.getPlayer(tabIndex);
if ~isempty(player) && isvalid(player)
    stop(player); % Detener el reproductor actual
end
player = audioplayer(audioProcesado, FsNuevo);
app.setPlayer(tabIndex, player);

```

4.4.4. Modulación

Las funciones de Modulación y Demodulación permiten transformar las señales de audio mediante diferentes técnicas de modulación, lo que es esencial para aplicaciones en comunicaciones. Estas funcionalidades se implementan mediante los métodos `applyModulation` y `applyDemodulation`.

Para modular la señal, lo primero es generar la señal portadora, lo que se hace utilizando la frecuencia de portadora especificada por el usuario.

```

% Crear la señal portadora
t = (0:length(audioData)-1) / Fs;
portadora = cos(2 * pi * fc * t)';

```

Se han implementado distintos tipos de modulación. Se emplea una estructura `switch` para manejarlos. Cada caso implementa una técnica de modulación específica utilizando funciones nativas de Matlab para asegurar precisión y eficiencia. A continuación, se muestra un ejemplo de las operaciones matemáticas realizadas para los tres primeros tipos de modulación:

```

switch tipoModulacion
case 'AM'
    if isempty(indiceAM) || indiceAM <= 0 || indiceAM > 1
        uialert(app.UIFigure, 'Especifique un índice de
modulación válido para AM (0 < índice ≤ 1).', 'Error');
        return;
    end
    % Señal AM
    modulatedSignal = (1 + indiceAM * audioData) .*
portadora;

case 'FM'
    if isempty(deltaF) || deltaF <= 0
        uialert(app.UIFigure, 'Especifique una desviación de
frecuencia válida para FM.', 'Error');
        return;
    end
    % Señal FM
    modulatedSignal = cos(2 * pi * fc * t' + 2 * pi * deltaF
* cumsum(audioData) / Fs);

case 'PM'
    % Señal PM
    modulatedSignal = cos(2 * pi * fc * t' + audioData);

```

La demodulación se adapta al tipo de modulación aplicado previamente, utilizando técnicas como la transformada de Hilbert para recuperar la señal original. Usamos las funciones `hilbert`, `diff`, `unwrap`, `angle`, etc. de Matlab. A continuación se muestra un ejemplo de la lógica de demodulación para los tres primeros tipos:

```

switch tipoModulacion
case 'AM'
    % Demodulación AM
    demodulatedSignal = abs(hilbert(audioData));

case 'FM'
    % Demodulación FM
    demodulatedSignal =
diff(unwrap(angle(hilbert(audioData)))) / (2 * pi * fc / Fs);

case 'PM'
    % Demodulación PM
    demodulatedSignal = angle(hilbert(audioData)) - 2 * pi *
fc * t';

```

4.4.5. Cuantificación

La cuantificación convierte una señal analógica en una señal digital al discretizar sus niveles de amplitud. Tras actualizar la pestaña activa, obtener los datos del audio y tomar los parámetros de cuantificación de la GUI, aplicamos la lógica de cuantificación.

La señal se normaliza y se escala según el número de niveles determinados por los bits de cuantificación. El redondeo (`round`) asegura que cada muestra se ajuste al nivel más cercano disponible, implementando así la cuantización.

```
function applyQuantization(app)
    % . . . (Resto del código)

    % Cuantización
    maxAmplitude = max(abs(audioData)); % Escalar
    levels = 2^numBits; % Niveles de cuantización
    quantizedData = round((audioData / maxAmplitude) * (levels / 2 -
1)) * (maxAmplitude / (levels / 2 - 1));
```

Luego, se normaliza la señal cuantificada para evitar cualquier distorsión causada por niveles de amplitud excesivos, asegurando que la señal permanezca dentro de un rango adecuado.

```
% Normalizar la señal cuantificada para evitar clipping
quantizedData = quantizedData / max(abs(quantizedData));
```

La señal cuantificada se almacena internamente y las gráficas se actualizan para reflejar los cambios, permitiendo al usuario visualizar el impacto de la cuantificación en tiempo real.

```
% Actualizar los datos de la señal cuantificada
app.setAudioData(tabIndex, quantizedData);

% Detener cualquier reproducción activa y asignar el reproductor
player = app.getPlayer(tabIndex);
if ~isempty(player) && isvalid(player)
    stop(player);
end

player = audioplayer(quantizedData, Fs);
app.setPlayer(tabIndex, player);
```

4.4.6. Filtro de reconstrucción

El Filtro de Reconstrucción es utilizado para suavizar una señal digitalizada o remuestreada, eliminando componentes de alta frecuencia que puedan haber sido introducidas durante el procesamiento.

Se limita el número de puntos a procesar para evitar un uso excesivo de memoria, garantizando que la aplicación permanezca responsiva incluso con señales largas.

```
function applyReconstructionFilter(app)
    % . . . (Resto del código)

    % Limitar el número máximo de puntos en el eje reconstruido
    maxPoints = 1e6; % Máximo permitido (1 millón de puntos)
    tReconstructed = linspace(0, max(tOriginal), min(maxPoints, N *
10));
```

Se emplea `interp1` con diferentes métodos de interpolación ('previous' para Sample & Hold y 'linear' para interpolación lineal), adaptando la técnica según el tipo de reconstrucción requerido.


```

        % Aplicar el filtro seleccionado
        switch filterType
            case 'Sample & Hold'
                % Filtro Sample & Hold
                reconstructedSignal = interp1(tOriginal, audioData,
tReconstructed, 'previous');

            case 'Interpolación lineal'
                % Filtro con interpolación lineal
                reconstructedSignal = interp1(tOriginal, audioData,
tReconstructed, 'linear');

            otherwise
                uialert(app.UIFigure, 'Tipo de filtro no reconocido.',
'Error');
                return;
        end

```

La señal reconstruida se normaliza para mantener niveles de amplitud consistentes y se guarda en un archivo temporal para facilitar su manejo posterior.

```

        % Normalizar la señal reconstruida
        reconstructedSignal = reconstructedSignal /
max(abs(reconstructedSignal));

```

Finalmente, se carga el fichero de audio temporal y se actualizan las gráficas.

4.5. Efectos y fenómenos psico-acústicos

En esta sección se describen las funcionalidades implementadas para aplicar diversos efectos y fenómenos psico-acústicos a las señales de audio. Estos efectos permiten modificar las características sonoras de las señales, ofreciendo herramientas avanzadas para el análisis y procesamiento de audio.

4.5.1. Ruido

El método `applyNoise` permite añadir diferentes tipos de ruido a la señal de audio seleccionada. Esta funcionalidad es útil para simular entornos ruidosos o para realizar pruebas de robustez en sistemas de procesamiento de audio.

Dependiendo del tipo de ruido seleccionado, se generan las muestras correspondientes y se escalan según el nivel indicado. Luego, se suma el ruido a la señal original para obtener la señal ruidosa. Para ello, usamos las funciones `randn`, `cumsum`, `pinknoise`, `rand` y `randperm` de Matlab.

```

function applyNoise(app)
    % . . . (Resto del código)

    % Generar ruido del tipo seleccionado
    switch noiseType
        case 'Blanco'
            noise = randn(size(audioData));

```

```

        case 'Marrón'
            noise = cumsum(randn(size(audioData)));
        case 'Rosa'
            noise = pinknoise(length(audioData));
        case 'Uniforme'
            noise = rand(size(audioData)) * 2 - 1;
        case 'Impulsivo'
            impulseProb = 0.01; % Probabilidad de impulsos
            noise = zeros(size(audioData));
            numImpulses = round(impulseProb * numel(audioData));
            indices = randperm(numel(audioData), numImpulses);
            noise(indices) = rand(numImpulses, 1) * 2 - 1;
        otherwise
            uialert(app.UIFigure, 'Tipo de ruido no reconocido.',
'Error');
        return;
    end

```

Luego, se escala el ruido según la intensidad elegida por el usuario y se suma a la señal original, que es la que después se mostrará en las gráficas y en el reproductor.

```

% Escalar el ruido según el nivel especificado
noise = noiseLevel * noise;

% Añadir el ruido a la señal original
noisySignal = audioData + noise;

% Actualizar la señal con ruido en la pestaña activa
app.setAudioData(tabIndex, noisySignal);

```

4.5.2. Eco

La función `applyEcho` introduce un efecto de eco en la señal de audio, simulando reflexiones sonoras que se producen en entornos reales. Este efecto puede ser ajustado en términos de tiempo de retardo e intensidad, y se puede aplicar en cascada para múltiples reflexiones.

El tiempo de retardo especificado se convierte a muestras utilizando la frecuencia de muestreo.

```

function applyEcho(app)
    % . . . (Resto del código)

    % Convertimos delay a samples
    delaySamples = round((delayMs / 1000) * Fs);

    % Necesitamos que audioData sea un vector columna
    audioData = audioData(:);

```

Dependiendo de si se selecciona la opción de cascada, se añaden múltiples ecos con intensidades decrecientes, simulando reflejos sucesivos en un entorno acústico. El eco se implementa sumando la señal original con las sucesivas repeticiones.

```

        if cascadeEnabled
            % Eco en cascada
            repetitions = 3;
            totalLength = length(audioData) + repetitions * delaySamples;
            echoSignal = zeros(totalLength, 1);

            % Señal original
            echoSignal(1:length(audioData)) = audioData;

            % Añadimos cada repetición
            for i = 1:repetitions
                startIdx = i * delaySamples + 1;
                endIdx = min(startIdx + length(audioData) - 1,
                    totalLength);

                signalLength = endIdx - startIdx + 1;

                echoSignal(startIdx:endIdx) = echoSignal(startIdx:endIdx)
+ ...
                    (intensity^i) * audioData(1:signalLength);
            end
        else
            % Eco simple
            totalLength = length(audioData) + delaySamples;
            echoSignal = zeros(totalLength, 1);

            % Señal original
            echoSignal(1:length(audioData)) = audioData;

            % Señal retardada
            delayedPortion = length(audioData);
            echoSignal(delaySamples + 1:delaySamples + delayedPortion) =
...
                echoSignal(delaySamples + 1:delaySamples +
delayedPortion) + ...
                    intensity * audioData;
        end
    end
end

```

Se normaliza la señal resultante para evitar distorsiones o clipping, manteniendo la integridad de la señal modificada.

```

% Normalizamos para evitar clipping
echoSignal = echoSignal / max(abs(echoSignal));

```

4.5.3. Inversión temporal

La función `invertSignal` invierte la señal de audio en el eje temporal, permitiendo analizar cómo se comporta la señal cuando se reproduce al revés. Este efecto es útil para estudios de análisis de señales y percepción sonora.

Utilizando la función `flipud`, se invierte la señal en el eje temporal, lo que resulta en una reproducción de la señal al revés.

```
function invertSignal(app)
    % . . . (Resto del código)

    % Invertir la señal en el tiempo
    invertedAudioData = flipud(audioData);
    app.setAudioData(tabIndex, invertedAudioData);
```

Se detiene cualquier reproducción activa y se crea un nuevo objeto `audioplayer` con la señal invertida para asegurar una reproducción coherente.

```
% Detener y reemplazar el reproductor
player = app.getPlayer(tabIndex);
if ~isempty(player) && isvalid(player)
    stop(player);
end

player = audioplayer(invertedAudioData, Fs);
app.setPlayer(tabIndex, player);

timeAxes = app.getTimeAxes(tabIndex);
cla(timeAxes);
```

Finalmente, se actualizan las gráficas correspondientes en el visualizador.

4.5.4. Efectos sonoros

Esta herramienta permite al usuario aplicar diversas transformaciones a las señales de audio para modificar su carácter y percepción auditiva. Estos efectos incluyen reverberación, efecto Haas y enmascaramiento.

El método principal de este apartado es `applyEffect`. Se utiliza una estructura `switch`, se determina qué efecto ha seleccionado el usuario y se llama a la función correspondiente (`applyReverb`, `applyHaasEffect` o `applyMasking`). Si no se ha seleccionado ningún efecto, se devuelve un mensaje de error.

```
function applyEffect(app)

    switch efectoSeleccionado
        case 'Reverberación'
            audioProcesado = app.applyReverb(audioData, Fs,
            app.DuracinsEditField.Value, app.IntensidadEditField.Value);

            case 'Efecto Haas'
                audioProcesado = app.applyHaasEffect(audioData, Fs,
                app.DesfasedecanalHaasEditField.Value);

            case 'Enmascaramiento'
                audioProcesado = app.applyMasking(audioData, Fs);

            otherwise
                uialert(app.UIFigure, 'Efecto no reconocido.', 'Error');
                return;
    end
```

Para aplicar la reverberación se hace una llamada al método `applyReverb`. Este método genera una respuesta al impulso con decadencia exponencial para simular la reverberación.

```
function audioOut = applyReverb(app, audioData, Fs, duration,
intensity)
    % . . . (Resto de Código)

    % Caída exponencial
    impulseResponse = exp(-3 * (0:reverbTime-1) / reverbTime)';
```

Luego, se normaliza y se mezcla la señal original con la respuesta al impulso mediante convolución.

```
% Normalizamos y escalamos por la intensidad
impulseResponse = impulseResponse / max(abs(impulseResponse)) *
intensity;

% Aplicamos la reverberacion usando la convolucion
audioOut = conv(audioData, impulseResponse, 'same');
```

Finalmente se mezcla la señal original con la reverberada y se normaliza para evitar clipping.

```
% Mezclamos las señales
audioOut = (1 - intensity) * audioData + intensity * audioOut;

% Normalizamos la salida
audioOut = audioOut / max(abs(audioOut));
end
```

Para aplicar el efecto Haas, usamos el método `applyHaasEffect`. Convertimos el retardo de ms a muestras según la fs.

```
function audioOut = applyHaasEffect(app, audioData, Fs, delayMs)
    % . . . (Resto de código)

    % Convertir el retardo de milisegundos a muestras
    delaySamples = round((delayMs / 1000) * Fs);

    % Crear la salida de audio
    audioOut = audioData;
```

Luego, introducimos el retardo en el canal derecho para lograr una percepción de espacialidad.

```
% Aplicar el retardo al canal derecho
audioOut(:, 2) = [zeros(delaySamples, 1); audioData(1:end-
delaySamples, 2)];
```

Finalmente, el enmascaramiento se realiza a través del método `applyMasking`. Se genera ruido rosa filtrando el ruido blanco con un filtro específico.

```
function audioOut = applyMasking(app, audioData, Fs)
    % . . . (Resto del código)

    % Ruido blanco
    whiteNoise = randn(N, 1);

    % Ruido rosa
    b = [0.049922035, -0.095993537, 0.050612699, -0.004408786];
    a = [1, -2.494956002, 2.017265875, -0.522189400];
    pinkNoise = filter(b, a, whiteNoise);
```

Se normaliza el ruido y se escala proporcionalmente a la RMS de la señal original.

```
pinkNoise = pinkNoise / max(abs(pinkNoise));
signalRMS = rms(audioData);
noiseRMS = rms(pinkNoise);
scaledNoise = (signalRMS / noiseRMS) * pinkNoise;
intensity = 0.3;
end
```

Finalmente, se añade el ruido escalado a la señal original, y se normaliza la señal resultante para evitar clipping.

```
maskedSignal = audioData + (intensity * scaledNoise);
audioOut = maskedSignal / max(abs(maskedSignal));
```

4.5.5. Transcripción de voz a texto

La función `transcribirAudio` permite convertir una señal de audio en texto utilizando un modelo de transcripción automática de voz de la compañía OpenAI llamado *Whisper Large v3 turbo* y al que se accederá a través de la API de Huggingface, que ofrece un entorno para realizar la inferencia de modelos de inteligencia artificial en la nube. Este proceso facilita la interpretación y análisis de contenidos hablados dentro de la aplicación.

Se verifica que el usuario haya seleccionado un archivo y que este exista en la ruta especificada. En caso contrario, se muestra una alerta informando al usuario.

```
function transcribirAudio(app)
    % . . . (Resto del código)

    % Obtener el archivo de audio de la ruta seleccionada
    filename = app.EditField.Value;

    if isempty(filename) || ~isfile(filename)
        uialert(app.UIFigure, 'Por favor, seleccione un archivo de audio válido.', 'Error');
        return;
    end
```

Se define la URL de la API de Huggingface que utiliza el modelo *Whisper* para la transcripción. Además, se configuran los encabezados HTTP necesarios para la autenticación mediante un token bearer.

```

        % URL y headers de la API
        API_URL = 'https://api-
inference.huggingface.co/models/openai/whisper-large-v3-turbo';
        headers = matlab.net.http.HeaderField('Authorization',
app.bearer);

```

El archivo de audio se lee en formato binario y se prepara una solicitud HTTP POST que incluye los datos de audio en el cuerpo del mensaje.

```

        % Leer los datos del archivo de audio
        fid = fopen(filename, 'rb');
        audioData = fread(fid, '*uint8');
        fclose(fid);

        % Crear solicitud HTTP
        request = matlab.net.http.RequestMessage('POST', headers,
matlab.net.http.MessageBody(audioData));

```

Se envía la solicitud a la API y se verifica el código de estado de la respuesta. Si la transcripción es exitosa (`StatusCode.OK`), se extrae el campo `text` de la respuesta y se muestra en la interfaz de usuario. En caso contrario, se notifica al usuario sobre el error.

```

try
    % Realizar la solicitud
    uri = matlab.net.URI(API_URL);
    response = send(request, uri);

    % Ocultar la animación de carga
    app.CargandoLabel.Visible = false;
    % Procesar la respuesta
    if response.StatusCode == matlab.net.http.StatusCode.OK
        responseData = response.Body.Data;
        if isfield(responseData, 'text')
            % Mostrar la transcripción en el campo de texto
            app.TextArea.Value = responseData.text;
        else
            uialert(app.UIFigure, 'No se pudo obtener la
transcripción.', 'Error');
        end
    else
        uialert(app.UIFigure, 'Error en la solicitud a la API de
Huggingface.', 'Error');
    end
end

```

4.5.6. Compresión/Expansión en el dominio temporal

Finalmente, la compresión y expansión del audio en el dominio temporal se realiza mediante la función `applyComprimirExpandir`. Se utiliza un factor de compresión/expansión que es equivalente a la velocidad de reproducción, de modo que, si este factor es mayor a 1, el audio se reproducirá más rápido. Si el factor es menor a 1, el audio se reproducirá con menor velocidad.

Primero definimos el tiempo original y calculamos la nueva duración y el número de muestras según el factor elegido.

```
function applyComprimirExpandir(app)
% . . . (Resto del Código)

% Definir el tiempo original
t_original = (0:length(audioData)-1) / Fs;

% Calcular la nueva duración y número de muestras
duracionOriginal = length(audioData) / Fs;
duracionNueva = duracionOriginal * (1/factor);
numMuestrasNueva = round(duracionNueva * Fs);
t_new = linspace(0, duracionOriginal, numMuestrasNueva)';
```

Luego, empleamos la interpolación para obtener un nuevo audio con el procesamiento para la compresión o la expansión en el tiempo.

```
% Interpolación para obtener el audio procesado
audioProcesado = interp1(t_original, audioData, t_new, 'linear',
0);
```

Una vez tenemos el audio procesado, actualizamos los datos de audio, creamos un nuevo reproductor para escuchar la señal procesada y actualizamos las gráficas del visualizador.

```
% Actualizar los datos de audio en la pestaña activa
app.setAudioData(app.tabIndex, audioProcesado);
% La frecuencia de muestreo permanece igual
% app.setFs(app.tabIndex, Fs); % No es necesario actualizar

% Detener cualquier reproducción activa
player = app.getPlayer(app.tabIndex);
if ~isempty(player) && isValid(player)
    stop(player);
end

% Crear un nuevo reproductor de audio con la señal procesada
player = audioplayer(audioProcesado, Fs);
app.setPlayer(app.tabIndex, player);

% Actualizar las gráficas de tiempo, frecuencia y espectrograma
timeAxes = app.getTimeAxes(app.tabIndex);
freqAxes = app.getFreqAxes(app.tabIndex);
specAxes = app.getSpecAxes(app.tabIndex);
app.updatePlot(timeAxes, freqAxes, specAxes);

% Actualizar la información de audio en la interfaz
app.updateAudioInfo(audioProcesado, Fs);
```


4.6. Comparación de espectrogramas

Se ha añadido en la interfaz un botón para ofrecer al usuario la posibilidad de comparar visualmente los espectrogramas de dos señales de audio distintas en una nueva ventana de la interfaz.

Primero, creamos la ventana y agregamos dos botones para cargar las dos señales que deseamos comparar.

```
function CompareSpectrograms(app)
    % Crear una nueva ventana (figure) con tamaño ajustado
    % Se reduce aún más la altura de la ventana
    fig = uifigure('Name', 'Comparar Espectrogramas', 'Position',
[100, 100, 1400, 700]);

    % Crear botones para cargar las dos señales
    btnLoad1 = uibutton(fig, 'push', ...
        'Text', 'Cargar Señal 1', ...
        'Position', [50, 650, 150, 30], ...
        'ButtonPushedFcn', @(btn,event) loadSignal(app, fig, 1));

    btnLoad2 = uibutton(fig, 'push', ...
        'Text', 'Cargar Señal 2', ...
        'Position', [220, 650, 150, 30], ...
        'ButtonPushedFcn', @(btn,event) loadSignal(app, fig, 2));
```

Luego, se crean los ejes de tiempo y espectrograma para cargar las señales en dos columnas paralelas. Se agrega una cuadrícula a la señal en el tiempo y se utiliza el mapa de color *jet* para el espectrograma. A continuación, se muestra un ejemplo del código empleado para la primera columna.

```
% Crear ejes para las señales organizados en dos columnas
% Columna 1: Señal 1
% Señal 1 - Tiempo
axTime1 = uiaxes(fig, 'Position', [50, 400, 600, 200]);
axTime1.Title.String = 'Señal 1 - Tiempo';
axTime1.XLabel.String = 'Tiempo (s)';
axTime1.YLabel.String = 'Amplitud';
grid(axTime1, 'on');

% Señal 1 - Espectrograma
axSpec1 = uiaxes(fig, 'Position', [50, 150, 600, 200]);
axSpec1.Title.String = 'Señal 1 - Espectrograma';
axSpec1.XLabel.String = 'Tiempo (s)';
axSpec1.YLabel.String = 'Frecuencia (Hz)';
colormap(axSpec1, jet);
```

Finalmente, almacenamos los ejes en la figura y almacenamos también las señales cargadas.

```
% Almacenar los ejes en la figura para accederlos posteriormente
fig.UserData.axTime1 = axTime1;
fig.UserData.axSpec1 = axSpec1;
fig.UserData.axTime2 = axTime2;
fig.UserData.axSpec2 = axSpec2;
```

```
% Almacenar las señales cargadas
fig.UserData.signal1 = [];
fig.UserData.Fs1 = [];
fig.UserData.signal2 = [];
fig.UserData.Fs2 = [];
end
```

5. Conclusiones

En esta práctica, hemos logrado desarrollar una interfaz gráfica de usuario (GUI) en MATLAB que facilita el análisis, procesamiento, grabación y reproducción de señales de audio de manera eficiente y accesible. La aplicación integra diversas funcionalidades avanzadas, como la visualización en dominios temporal y frecuencial, filtrado digital, remuestreo, modulación, cuantificación y la aplicación de efectos psico-acústicos, proporcionando a los usuarios una herramienta completa para el manejo de señales de audio.

El diseño modular del código, utilizando métodos Get/Set y una estructura organizada de propiedades privadas, ha permitido una gestión efectiva de los datos y una navegación fluida entre las diferentes funcionalidades de la interfaz. La implementación de callbacks y eventos asegura una interacción dinámica y responsiva, mejorando significativamente la experiencia del usuario.

Uno de los logros destacados es la integración de la transcripción de voz a texto mediante la API de Hugging Face, lo que añade una dimensión adicional de utilidad a la aplicación al permitir la conversión automática de audio a texto.

Durante el desarrollo, hemos tenido desafíos relacionados con la optimización de recursos y la implementación de una interfaz intuitiva sin sacrificar la potencia de las herramientas ofrecidas. La gestión de errores mediante bloques try-catch y alertas de usuario ha contribuido a aumentar la robustez y fiabilidad de la aplicación, asegurando que los usuarios reciban retroalimentación clara en caso de incidencias.

Para futuras mejoras, se podría considerar la ampliación de la compatibilidad con más formatos de audio, la incorporación de algoritmos de procesamiento adicionales y la optimización de la interfaz para manejar proyectos de mayor complejidad.

En resumen, esta práctica ha permitido desarrollar una herramienta versátil y potente para el procesamiento de audio en MATLAB, combinando facilidad de uso con funcionalidades avanzadas. La experiencia adquirida durante este proyecto sienta las bases para futuros desarrollos en el ámbito del análisis de señales y la ingeniería de audio, destacando la importancia de un diseño bien estructurado y orientado al usuario.

6. Anexos

1. **Código Fuente Completo:** El código fuente completo del proyecto, incluyendo los scripts, funciones y clases utilizadas para el desarrollo de la aplicación, se encuentra disponible en la carpeta proporcionada como parte de esta entrega en el campus virtual de la Universidad de Valladolid para la asignatura de Fundamentos de Imagen y Sonido.
2. **Memoria de Funcionamiento /Guía de Usuario:** La memoria de funcionamiento detallada o guía de usuario completa, que describe el uso de la interfaz y las funcionalidades disponibles, está incluida en la misma carpeta de la entrega mencionada anteriormente.
3. **Capturas de Pantalla y Videos:** Se adjuntan capturas de pantalla de la aplicación en funcionamiento, así como videos demostrativos que muestran la interfaz de usuario y sus diferentes funcionalidades en acción. Estos recursos visuales se encuentran disponibles en la carpeta correspondiente a la entrega.

7. Referencias

1. **MathWorks.** *Documentación de App Designer en MATLAB*. MathWorks, 2024.
Disponible en: <https://es.mathworks.com/help/matlab/app-designer.html>
2. **MathWorks.** *MATLAB Audio Toolbox Getting Started Guide*. MathWorks, 2024.
Disponible en: <https://es.mathworks.com/help/audio/>
3. **MathWorks.** *MATLAB Audio Toolbox Reference*. MathWorks, 2024.
Disponible en: <https://es.mathworks.com/help/audio/>
4. **MathWorks.** *MATLAB Audio Toolbox User's Guide*. MathWorks, 2024.
Disponible en: <https://es.mathworks.com/help/audio/>
5. **MathWorks.** *MATLAB App Building 2019*. MathWorks, 2019.
Disponible en: <https://es.mathworks.com/help/matlab/app-designer.html>
6. **MathWorks.** *MATLAB App Building 2020*. MathWorks, 2020.
Disponible en: <https://es.mathworks.com/help/matlab/app-designer.html>
7. **MathWorks.** *MATLAB Signal Processing Toolbox User's Guide 2018*. MathWorks, 2018.
Disponible en: <https://es.mathworks.com/help/signal/>
8. **MathWorks.** *MATLAB Signal Processing Toolbox User's Guide 2019*. MathWorks, 2019.
Disponible en: <https://es.mathworks.com/help/signal/>
9. **Hugging Face.** *API Inference: Automatic Speech Recognition (ASR)*. Hugging Face, 2024.
Disponible en: <https://huggingface.co/docs/api-inference/tasks/automatic-speech-recognition>